Architecture 1001: x86-64 Assembly TooManyParameters.c

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

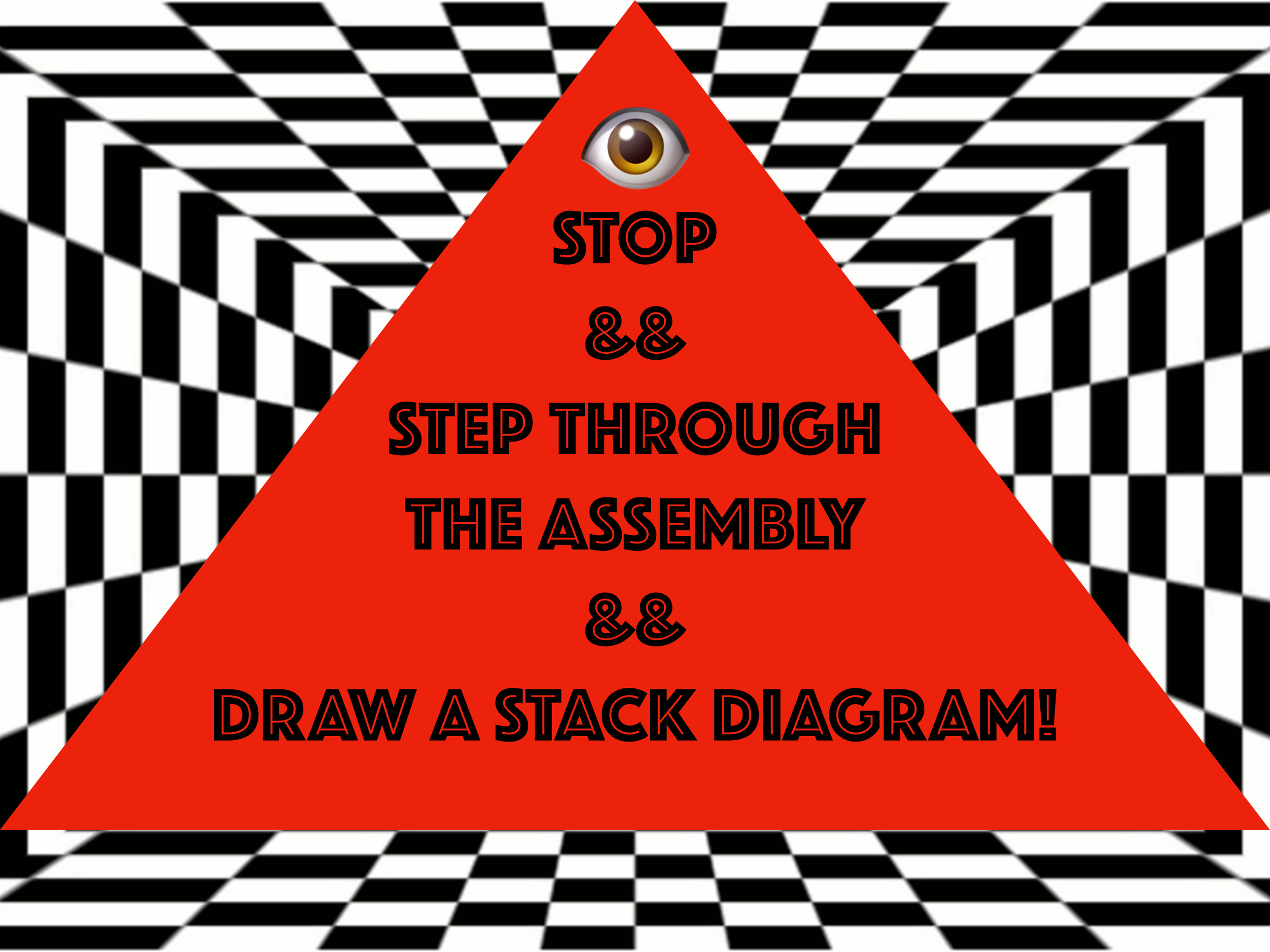
Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

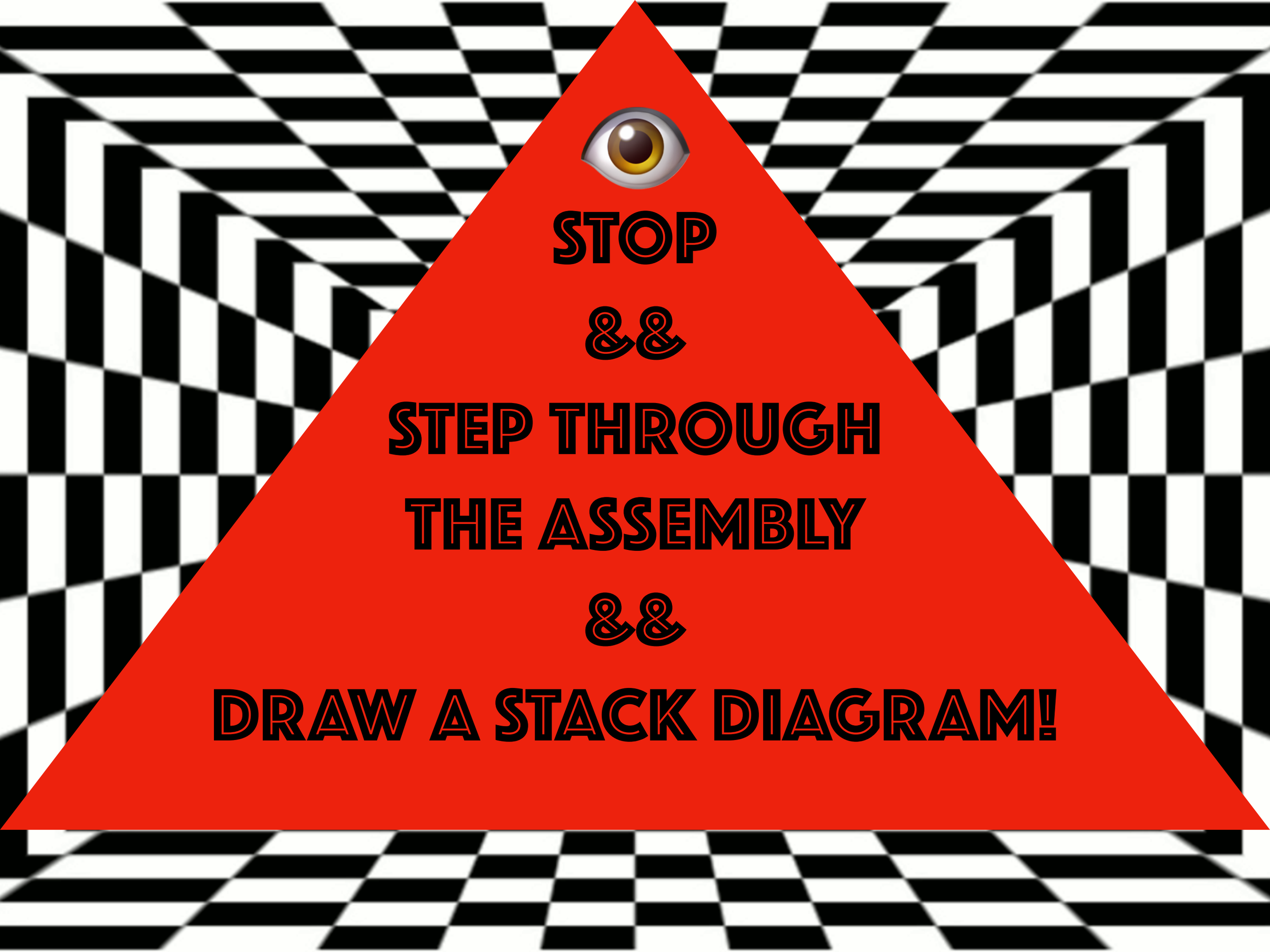
TooManyParameters.c

```c
#define uint64 unsigned long long

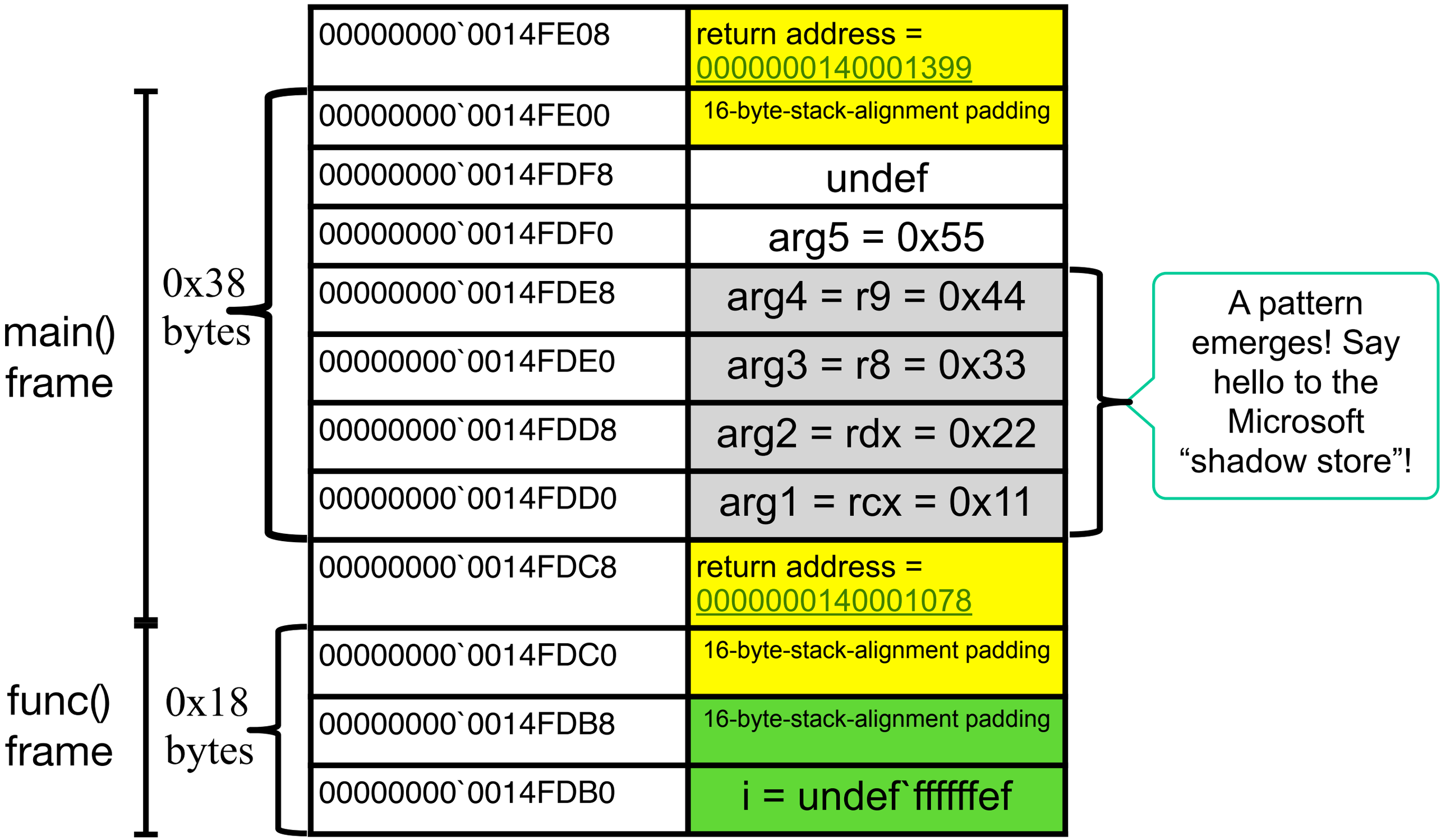
int func(uint64 a, uint64 b, uint64 c, uint64 d, uint64 e){
    int i = a+b-c+d-e;
    return i;
}
int main(){
    return func(0x11,0x22,0x33,0x44,0x55);
}
```

```
func:
0000000140001000  mov  qword ptr [rsp+20h],r9
0000000140001005  mov  qword ptr [rsp+18h],r8
000000014000100A  mov  qword ptr [rsp+10h],rdx
000000014000100F  mov  qword ptr [rsp+8],rcx
0000000140001014  sub  rsp,18h
0000000140001018  mov  rax,qword ptr [rsp+28h]
000000014000101D  mov  rcx,qword ptr [rsp+20h]
0000000140001022  add  rcx,rax
0000000140001025  mov  rax,rcx
0000000140001028  sub  rax,qword ptr [rsp+30h]
000000014000102D  add  rax,qword ptr [rsp+38h]
0000000140001032  sub  rax,qword ptr [rsp+40h]
0000000140001037  mov  dword ptr [rsp],eax
000000014000103A  mov  eax,dword ptr [rsp]
000000014000103D  add  rsp,18h
0000000140001041  ret
main:
0000000140001050  sub  rsp,38h
0000000140001054  mov  qword ptr [rsp+20h],55h
000000014000105D  mov  r9d,44h
0000000140001063  mov  r8d,33h
0000000140001069  mov  edx,22h
000000014000106E  mov  ecx,11h
0000000140001073  call 0000000140001000
0000000140001078  add  rsp,38h
000000014000107C  ret
```


STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
DRAW A STACK DIAGRAM!


STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
DRAW A STACK DIAGRAM!

The stack looks like this at line 0000000140001031 in func():



"The x64 Application Binary Interface (ABI) uses a four-register *fast-call* calling convention by default. Space is allocated on the call stack as a ***shadow store*** for callees to save those registers."

"The caller must always allocate sufficient space to store four register parameters, even if the callee doesn't take that many parameters."

"Any parameters beyond the first four must be stored on the stack after the shadow store before the call."

“Shadow store” reference https://docs.microsoft.com/en-us/cpp/build/x64-calling-convention?view=msvc-160

"Who knows what evil lies in the ~~hearts~~ stacks of ~~men~~ compilers?"

"Who knows what evil lies in the ~~hearts~~ stacks of ~~men~~ compilers?"



TooManyParameters.c takeaways

- Microsoft compiler specifically implements a "calling convention" that not only passes the first 4 arguments through registers, but also reserves “shadow store” for them on the stack.
- “The callee has the responsibility of dumping the register parameters into their ***shadow space*** if needed.”
- Compiler reserves this space even if no function parameters are passed to another function

```
func:
mov  qword ptr [rsp+20h],r9
mov  qword ptr [rsp+18h],r8
mov  qword ptr [rsp+10h],rdx
mov  qword ptr [rsp+8],rcx
sub  rsp,18h
mov  rax,qword ptr [rsp+28h]
mov  rcx,qword ptr [rsp+20h]
add  rcx,rax
mov  rax,rcx
sub  rax,qword ptr [rsp+30h]
add  rax,qword ptr [rsp+38h]
sub  rax,qword ptr [rsp+40h]
mov  dword ptr [rsp],eax
mov  eax,dword ptr [rsp]
add  rsp,18h
ret
main:
sub  rsp,38h
mov  qword ptr [rsp+20h],55h
mov  r9d,44h
mov  r8d,33h
mov  edx,22h
mov  ecx,11h
call 0000000140001000
add  rsp,38h
ret
```

```c
#define uint64 unsigned long long

int func(uint64 a, uint64 b, uint64 c, uint64 d, uint64 e){
    int i = a+b-c+d-e;
    return i;
}
int main(){
    return func(0x11,0x22,0x33,0x44,0x55);
}
```